

Scheduling

CS232 Operating Systems, Fall 2019

Taj Muhammad Khan

*Figures, unless otherwise stated, are from OSTEP 1.0

Some definitions

- Scheduler: part of OS that selects the next process to run
- Scheduling policy: the policy/algorithm according to which the scheduler selects the next process
- Workload: the set of process that the OS has to run
 - Knowing the workload helps in developing better policies
- Metric: the criterion to measure the performance of a scheduler
 - No. of jobs/sec
 - T_{total}
 - $T_{turnaround} = T_{completion} - T_{arrival}$

Workload assumptions

1. Each job runs for the same amount of time
2. All jobs arrive at the same time
3. Once started, each job runs to completion
4. All jobs only use the CPU (i.e. they perform no I/O)
5. The runtime of each job is known

They may seem unrealistic (they actually are) but don't worry.

Policy: FIFO

- Three jobs: A, B, C
- Each runs for 10 secs
- Arrive at same time $t=0$

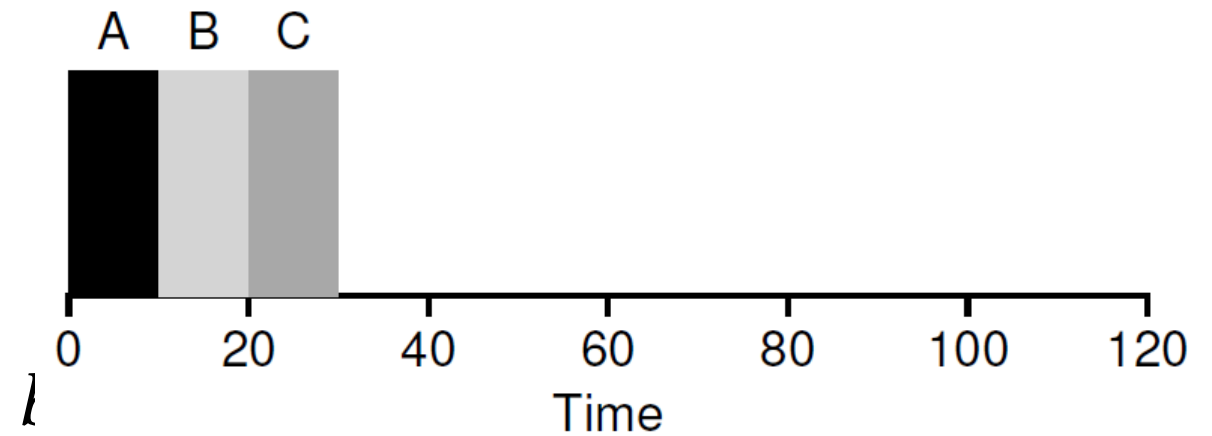


Figure 7.1: FIFO Simple Example

Policy: FIFO

Relax assumption no. 1!
Length of A = 100 secs

$$T_{turnaround} = \frac{100+110+120}{3} = 110$$

Small jobs have to wait a lot!

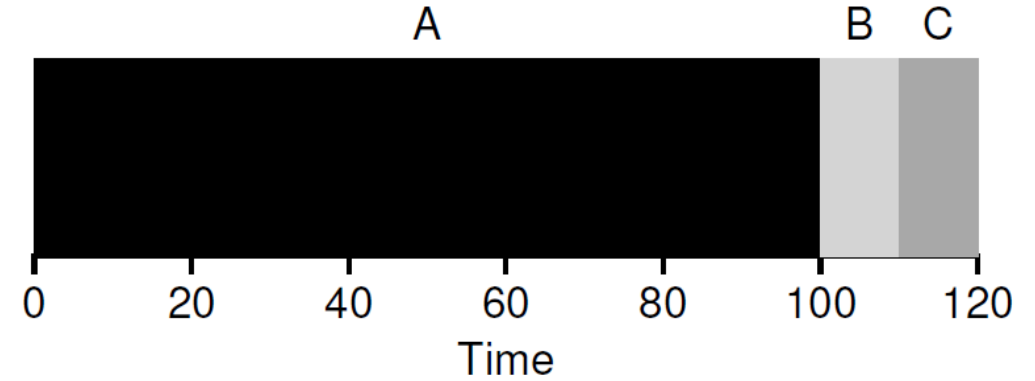


Figure 7.2: Why FIFO Is Not That Great

Policies: Shortest Job First (SJF)

$$T_{turnaround} = \frac{10+20+120}{3} = 50$$

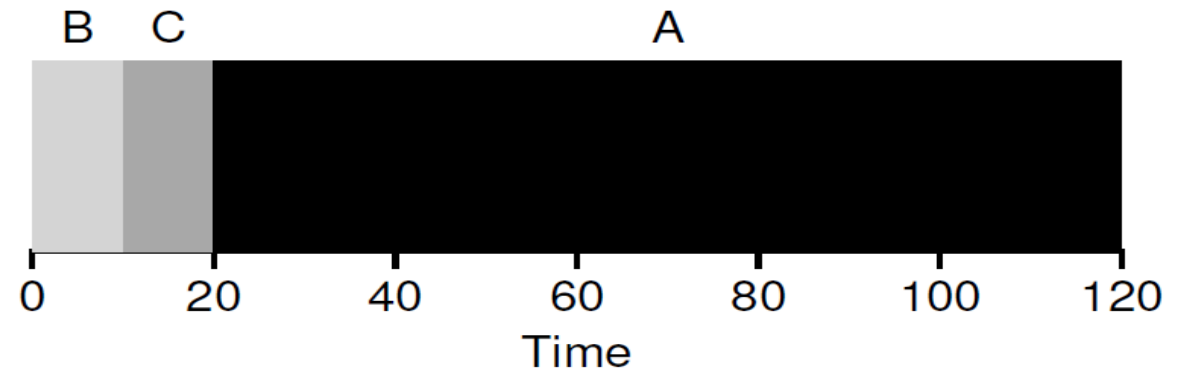


Figure 7.3: SJF Simple Example

Now relax assumption no. 2!

$$T_{turnaround} = \frac{100+(110-10)+(120-10)}{3} = 103.33$$

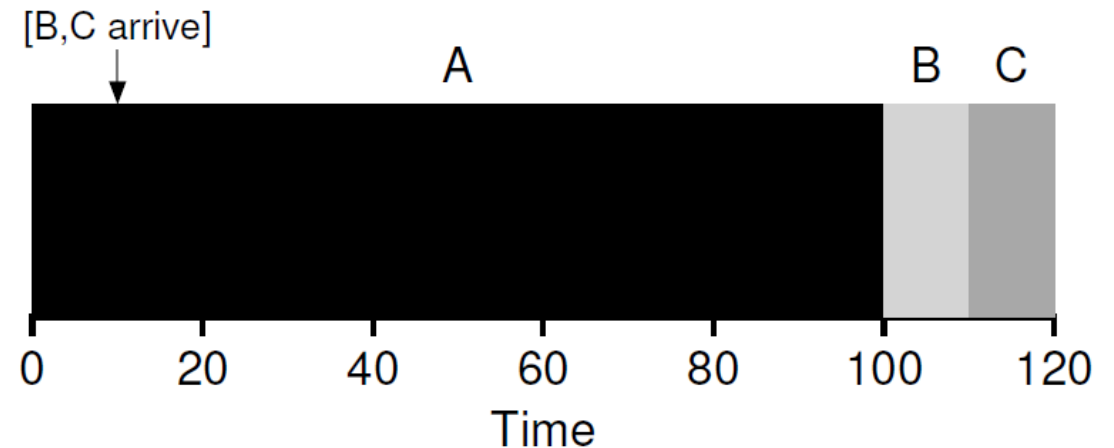


Figure 7.4: SJF With Late Arrivals From B and C

Policy: Shortest Time-to-Completion First (STCF)

- Let's now relax assumption no. 3
- Scheduler can now **preempt** running jobs!
- SJF was a non-preemptive policy
- STCF is also known as Preemptive SJF (PSJF)
- Whenever a new job arrives, the scheduler re-decides which job to run

$$T_{turnaround} = \frac{(20-10) + (30-10) + 120}{3} = 50$$

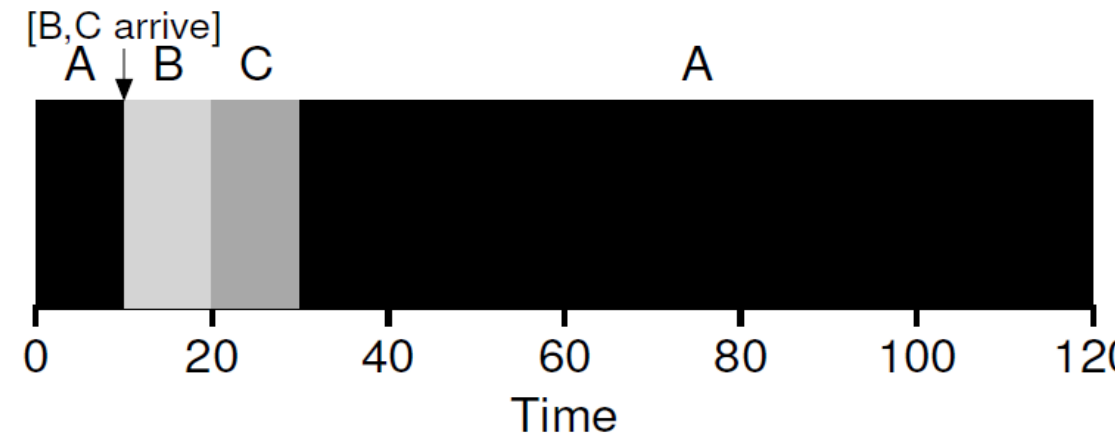


Figure 7.5: STCF Simple Example

Interactive Systems

- For batch computing, non-preemptive schedulers make sense as programs were run non-interactively
- Timeshared systems where users sit at a terminal and send commands to interact with the system, responsiveness is important
- A new metric was needed

$$T_{response} = T_{firstrun} - T_{arrival}$$

Metric: Response Time

- SJF, STCF result in non-optimal response times

- $T_{response} = \frac{0+5+10}{3} = 5$

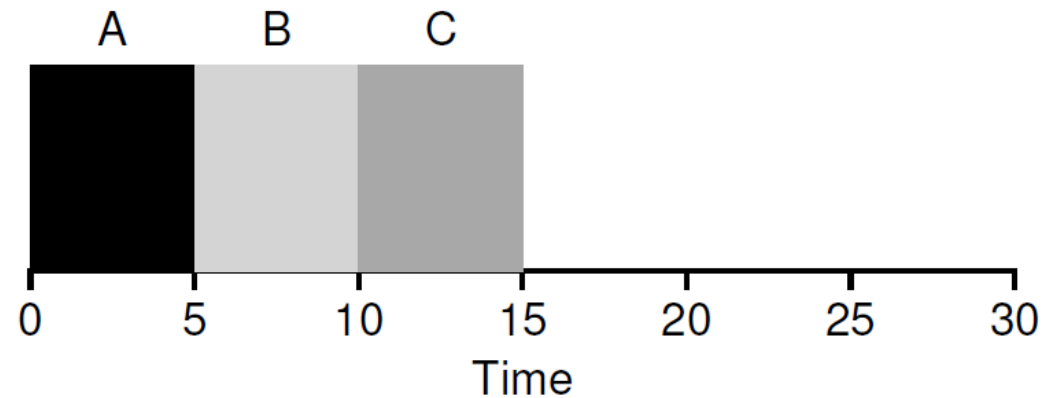


Figure 7.6: SJF Again (Bad for Response Time)

Policy: Round Robin

- A *time-slice* or *scheduling quantum* is defined
- A job is run for a time-slice and then the scheduler moves to next job

- $T_{response} = \frac{0+1+2}{3} = 1$

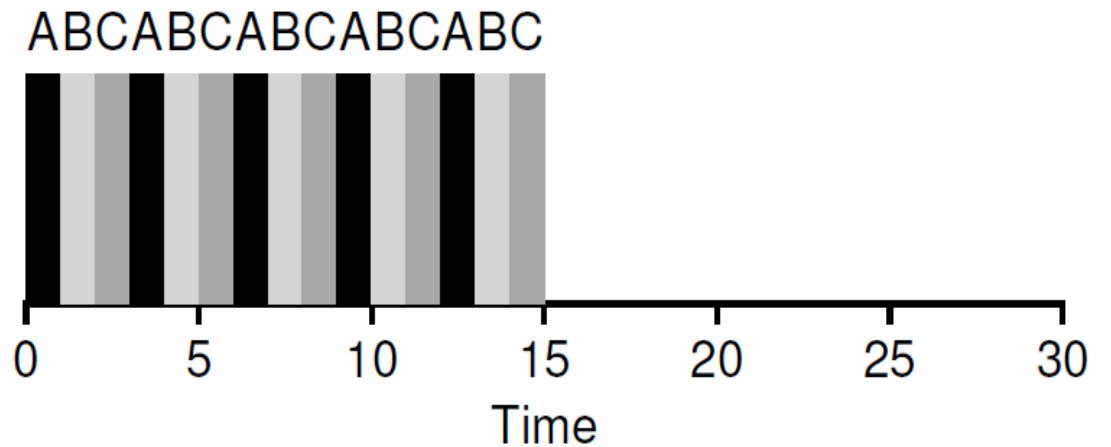


Figure 7.7: Round Robin (Good For Response Time)

Policy: Round Robin contd.

- Responsiveness is related to Fairness
- Round Robin is a Fair policy
- Fairness (Response time) is at odds with Performance (Turnaround time)
- The shorter the time-slice → the more responsive the system
- The shorter the time-slice → the less performant the system
 - Overhead incurred due to context switches
 - Overhead incurred due to cache/TLB invalidation

I/O

- Let's relax assumption no. 4
- Processes often do I/O
- Since I/O is slower, so they've to wait for the result
- They'll be blocked till the response arrives
- Keeping the CPU idle would be a waste of resources

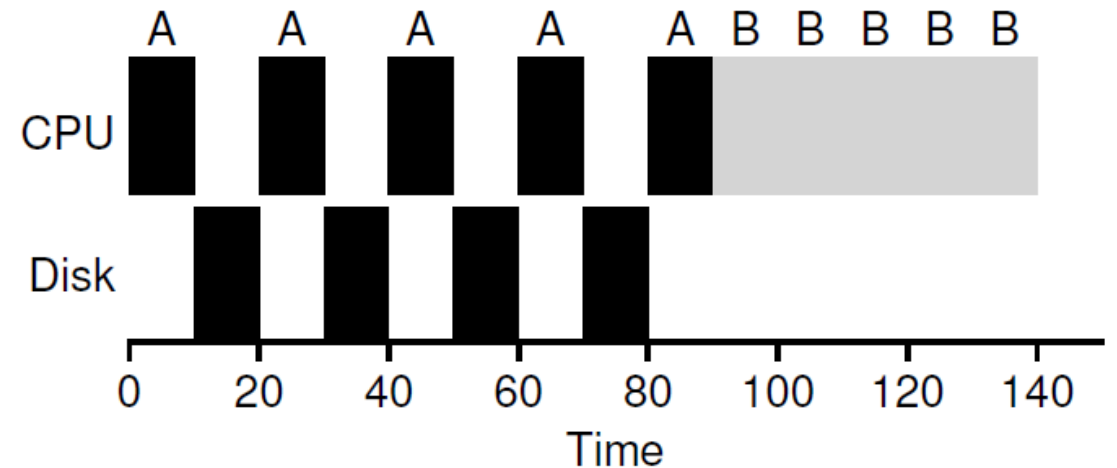


Figure 7.8: Poor Use Of Resources

Overlap process executions

- With preemptive schedulers we've the capability to context switch

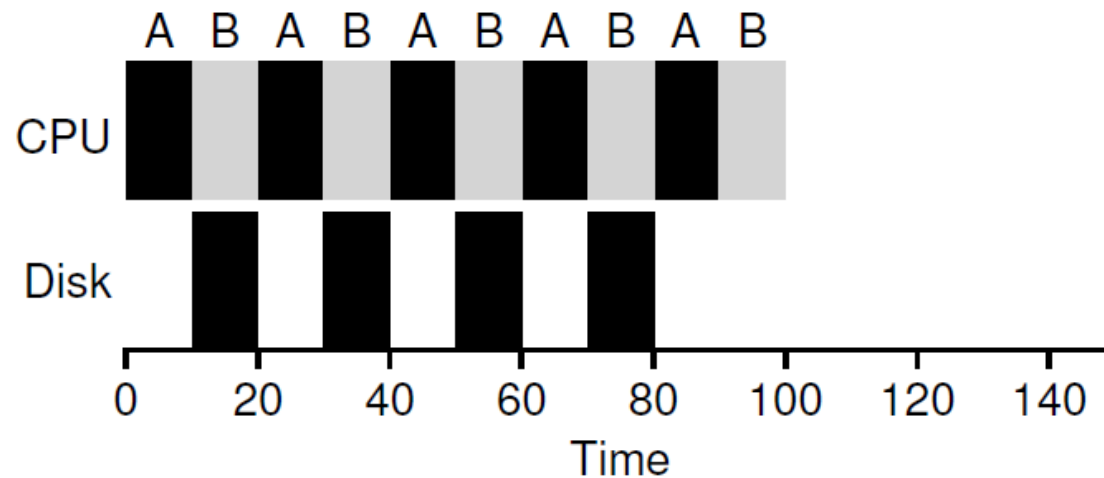


Figure 7.9: **Overlap Allows Better Use Of Resources**

Relax assumption no. 5